Simplified Setup — Step by Step

These commands work exactly the same on Windows (running WSL2) and on Mac.

> 📋 ***How to read this:*** *any command shown in a grey box is something you* ***type into your terminal and press Enter****.*

Follow each step in order. Read the short description first, then run the instruction. Only move on when a step finishes successfully.

You will move between just two folders. Each step below tells you which one to be in:

- `agentic-rag-n8n-cohort` — the main folder (the helper scripts live here).
- `agentic-rag-n8n-cohort/self-hosted-ai-starter-kit` — the stack folder (Docker runs from here).

> *Lost track of where you are? Type* `pwd` *and press Enter — it prints your current folder.*

Step 1 — Open the right terminal

You already have WSL2 installed. On **Windows** you must open the **WSL2 Linux terminal** — this is the **Ubuntu app**, **NOT** PowerShell and **NOT** Command Prompt. Most failed setups happen because the commands were typed into PowerShell, where they do not work the same way. On **Mac**, the built-in **Terminal** is the correct one. From this point on, every command is identical on both systems.

Windows: click **Start**, type **Ubuntu**, and open the **Ubuntu** app *(it may show as "Ubuntu" or "Ubuntu 24.04 LTS" — either is correct)*.

Mac: press **Cmd + Space**, type **Terminal**, and open it.

> *Check you are in the right place: your prompt should look like* `you@PC:~$` *(Linux),* ***not*** `PS C:\Users\you>` *(PowerShell).*

Step 2 — Install Docker Desktop and start it

Docker Desktop is the one program you install by hand — it runs the entire stack for you. After installing, start it and wait until the whale icon stops animating. On **Windows** you must also turn on **WSL Integration** so Docker can be used from your Ubuntu terminal.

1. Download and install: https://www.docker.com/products/docker-desktop/
2. Start Docker Desktop. Wait until the whale icon stops animating.
3. **Windows only:** Docker Desktop → **Settings → Resources → WSL Integration** → toggle **ON** your Ubuntu distro → **Apply & Restart**.

Step 3 — Clone the course repo

This downloads everything you need — the pinned stack, the helper scripts, and the sample files — into a new folder. Run it in the **same terminal window** from Step 1. The second line moves you inside the folder you just downloaded.

```
git clone https://github.com/SwarupSG/agentic-rag-n8n-cohort.git
```

```
cd agentic-rag-n8n-cohort
```

> *You are now inside the* ***`agentic-rag-n8n-cohort`*** *folder. Stay here for Step 4.*

> 💡 ***Recommended — work inside your IDE.*** *Now that the repo is cloned, open it in* ***VS Code or Antigravity (connected to WSL)*** *and run everything from its built-in* ***Terminal*** *panel — you'll see your files on the left, and that terminal is already the WSL shell (not PowerShell). From the* `agentic-rag-n8n-cohort` *folder, run* `code .` *(VS Code) or open the folder in Antigravity, then open the Terminal panel.*
>
> *Prefer a plain terminal? That's fine too — just keep using the WSL Ubuntu terminal (Mac: Terminal).*

Step 4 — Run the preflight check

This script confirms your machine is ready — Docker is reachable, you have enough disk space, and you are in the correct terminal — **before** you start anything. It prints a check mark for each test.

Folder: `agentic-rag-n8n-cohort` (Step 3 left you here). If you opened a new terminal since then, run `cd agentic-rag-n8n-cohort` first.

Option 1 (try this first):

```
./scripts/preflight.sh
```

Option 2 (use this if Option 1 says `Permission denied`):

```
bash scripts/preflight.sh
```

Continue only if it ends with **Preflight passed**. If it stops with an error, fix what it names, then run it again.

Step 5 — Create the .env file

The stack needs a small settings file named `.env`. It does not exist yet — only a template does. The first line below **moves you into the stack folder**; the second copies the template into the real file. Without it, the next step refuses to start.

Folder: `agentic-rag-n8n-cohort` (where Step 4 left you).

```
cd self-hosted-ai-starter-kit
cp .env.example .env
```

> *You are now inside the* ***`self-hosted-ai-starter-kit`*** *folder. Stay here for Steps 6, 7, and 8.*

Step 6 — Start the stack

This downloads and starts all four containers (n8n, Ollama, Qdrant, Postgres). The **first run pulls a few GB**, so give it a few minutes. Watch the output — **every line must end with `Started`**.

Folder: `self-hosted-ai-starter-kit` (where Step 5 left you).

```
docker compose --profile cpu up -d
```

> *Have an NVIDIA GPU? Use this line instead:*
>
> ```
> docker compose --profile gpu-nvidia up -d
> ```

If it stops partway with an error, the stack is half-started. Run these two lines, then continue:

```
docker compose --profile cpu down
docker compose --profile cpu up -d
```

Step 7 — Confirm Ollama is running

Before the next step you must be sure the Ollama container actually started. This command lists the containers that are running right now. You are looking for `ollama` in the list.

Folder: stay where you are — this works from any folder.

```
docker ps
```

If you do **not** see `ollama`, the next step will fail — go back and repeat Step 6 first.

Step 8 — Pull the two models

These two commands download the AI models into the Ollama container — one for chat, one for turning text into vectors. They run **inside** the container, which is why Ollama had to be running first (Step 7).

Folder: stay where you are — these work from any folder.

```
docker exec ollama ollama pull llama3.2
docker exec ollama ollama pull nomic-embed-text
```

Wait for each to print **success**.

Step 9 — Verify everything

This final script checks the whole setup end to end — all containers up, both models present, and n8n able to reach the other services. It is your proof that the environment is ready.

Folder: `self-hosted-ai-starter-kit` (where Steps 5–8 left you). The script lives one level up, so first move back to the main folder:

```
cd ..
```

> *You are now back in the* ***`agentic-rag-n8n-cohort`*** *folder.*

Option 1 (try this first):

```
./scripts/verify_setup.sh
```

Option 2 (use this if Option 1 says `Permission denied`):

```
bash scripts/verify_setup.sh
```

You are done when it prints **All checks passed**.

Step 10 — Open the apps

Your stack is now running on your own machine. Open these two pages in your normal web browser. **n8n** is where you will build everything; the **Qdrant** dashboard shows your vector database.

- n8n → http://localhost:5678
- Qdrant → http://localhost:6333/dashboard